

Practical Lab: From Autoencoders to Variational Autoencoders

Generative Models – M2

Duration: 2 hours, including setup and a short report
Dataset: MNIST
Hardware: CPU, or one small CUDA GPU if available

Most of the plotting and data-loading code is provided. You will work directly in two scripts: `autoencoder.py` and `vae.py`. Search for `TODO` in these files.

1. Setup

Clone the repository provided by the instructor and create the environment:

```
git clone https://github.com/SegoleneMartin/VAE_mnist_lab.git
conda env create -f environment.yml
conda activate genmodels-vae
python check_install.py
```

The environment is intentionally small, but the first installation can still take a few minutes because PyTorch and torchvision are large packages.

Download MNIST once:

```
python download_mnist.py
```

The dataset is stored in `data/` inside the cloned repository.

CPU/GPU selection. Every training command requires you to choose the execution device explicitly. Use `-gpu` if a CUDA GPU is available, or `-cpu` to run on your own computer without a GPU.

If you use a GPU, select it **once** at the beginning of the practical session (replace 0 by the GPU number assigned to you):

```
export CUDA_VISIBLE_DEVICES=0
```

The variable remains active for all commands launched from the same terminal. You can then simply run, for example,

```
python autoencoder.py --gpu --sampling-distribution normal
```

If you do not have access to a GPU, use `-cpu` instead. In the rest of the subject, GPU commands are shown by default; on a CPU, simply replace `-gpu` by `-cpu`.

2. Autoencoders: from reconstruction to generation

An autoencoder contains an encoder E_ϕ and a decoder D_θ :

$$z = E_\phi(x), \quad \hat{x} = D_\theta(z).$$

It is trained to minimize a reconstruction error,

$$\min_{\theta, \phi} \sum_i \|x_i - D_\theta(E_\phi(x_i))\|^2.$$

In this lab the latent space has dimension **2**, so we can visualize it directly.

Task 1 – Complete and train the autoencoder

Open `autoencoder.py` and complete **TODO 1–4**:

1. implement the forward pass: given an input image x , first compute its latent representation $z = E_\theta(x)$ with the encoder, then reconstruct the image as $\hat{x} = D_\theta(z)$ with the decoder, and return both z and $\hat{x}$;
2. implement the squared reconstruction loss (sum over pixels, mean over the batch);
3. write a standard PyTorch optimization step: zero gradients, forward pass, loss, backward pass, optimizer step.

Then run:

```
python autoencoder.py --cpu --sampling-distribution normal
```

This first run creates:

- `outputs/ae_reconstructions.png`,
- `outputs/ae_latent.png`,
- `outputs/ae_normal_samples.png`,
- `outputs/ae.pt`.

Inspect the reconstructions, latent space, and Gaussian samples before continuing. Increase the number of epochs to improve the results a bit.

Question 1. Compare the reconstructions and the samples obtained by drawing $z \sim \mathcal{N}(0, I_2)$. Why is $\mathcal{N}(0, I_2)$ not necessarily a good model of the latent codes learned by the autoencoder?

3. Learning the latent distribution with a Gaussian mixture

The autoencoder has produced latent codes $z_i = E_\phi(x_i)$, but the reconstruction objective did not specify how these codes should be distributed. We can therefore estimate their distribution *after* training. A mixture of K Gaussians has density

$$p_{\text{GMM}}(z) = \sum_{k=1}^K \pi_k \mathcal{N}(z; \mu_k, \Sigma_k).$$

We use `sklearn.mixture.GaussianMixture`, which performs the EM optimization for us.

Task 2 – Encode, fit, sample, decode

Complete **TODO 5** in `autoencoder.py`. Run the same script again, now changing the latent sampling distribution:

```
python autoencoder.py --cpu --sampling-distribution gmm
```

This reloads the AE checkpoint from Task 1 (it does not retrain it), fits the GMM, and creates `ae_latent_gmm.png` and `ae_gmm_samples.png`.

Question 2. Compare `ae_normal_samples.png` and `ae_gmm_samples.png`. Why can the fitted GMM produce more meaningful samples?

4. Variational autoencoder

Instead of fitting a latent distribution after training, a VAE learns a probabilistic latent representation jointly with the decoder. We use the generative model

$$z \sim \mathcal{N}(0, I_2), \quad x | z \sim \mathcal{N}(D_\theta(z), I).$$

The exact posterior $p_\theta(z \mid x)$ is generally intractable, so the encoder represents the approximate posterior

$$q_\phi(z \mid x) = \mathcal{N}\left(\mu_\phi(x), \text{diag}(\sigma_\phi(x)^2)\right).$$

Task 3 – Gaussian encoder

Open `vae.py`. Complete **TODO 1–2**:

1. create two linear layers mapping the 64-dimensional hidden representation to the 2-dimensional latent space: one for $\mu(x)$ and one for $\log \sigma^2(x)$;
2. use these two heads in the encoder.

Task 4 – Reparameterization trick

To backpropagate through the stochastic latent variable, use

$$\varepsilon \sim \mathcal{N}(0, I), \quad z = \mu + \sigma \odot \varepsilon.$$

Complete **TODO 3**. Useful PyTorch functions are `torch.exp(...)` and `torch.randn_like(...)`.

Task 5 – Compute and implement the VAE objective

According to the class, the ELBO objective that we maximize is

$$\text{ELBO}(\theta, \phi; x) = \mathbb{E}_{q_\phi(z \mid x)} [\log p_\theta(x \mid z)] - \text{KL}(q_\phi(z \mid x) \parallel p(z)). \quad (1)$$

We will admit that the KL divergence between two Gaussian distributions admits a closed-form expression. In our case, with

$$q_\phi(z \mid x) = \mathcal{N}(\mu, \text{diag}(\sigma^2)), \quad p(z) = \mathcal{N}(0, I),$$

we have

$$\text{KL}(q_\phi(z \mid x) \parallel p(z)) = \frac{1}{2} \sum_{i=1}^d \left(\sigma_i^2 + \mu_i^2 - 1 - \log \sigma_i^2 \right). \quad (2)$$

Implement **TODO 4–6**.

Train the VAE:

```
python vae.py --gpu
```

This produces reconstructions, samples from $z \sim \mathcal{N}(0, I)$, a plot of the latent means, training curves, a regular 2D decoder grid, and an interpolation between two digits.

Question 3. What is the role of the KL term? Why would forcing $q_\phi(z \mid x) = \mathcal{N}(0, I)$ exactly for every input x be undesirable?

Question 4. Why do we use the reparameterization $z = \mu + \sigma \varepsilon$ instead of simply writing “sample $z \sim q_\phi(z \mid x)$ ” inside the network?

5. Compare the two generative approaches

Compare the AE+GMM samples with the VAE samples.

Question 5. Is the VAE better than AE+GMM in your experiment? Explain why or why not. What is the conceptual advantage of the VAE approach? Finally, suggest one concrete change that you would try to improve the generated images.

Question 6 (Bonus). What do `vae_latent_grid.png` and `vae_interpolation.png` correspond to? What do they tell us?

Question 7 (Bonus). Retrain with another KL weight β . What does β do/change?

Useful commands

Fast debugging runs:

```
python autoencoder.py --cpu --sampling-distribution normal --quick  
python vae.py --cpu --quick
```

Clean generated outputs:

```
rm -rf outputs
```